

A Peer-to-Peer Global AI Computer

Akhilesh Thite

P2P Labs

athite@p2plabs.xyz

Abstract

Artificial intelligence has become increasingly centralized, requiring organizations and individuals to depend on remote inference providers despite rapid growth in locally available compute resources. At the same time, billions of CPUs, GPUs, NPUs, and other specialized accelerators remain significantly underutilized. This paper proposes the concept of a **Global AI Computer**, a decentralized architecture that aggregates heterogeneous devices into a unified AI infrastructure through peer-to-peer coordination, distributed model storage, capability-aware scheduling, and fault-tolerant execution.

Unlike conventional distributed inference systems that assume static clusters or centrally managed infrastructure, the proposed architecture treats participating devices as a dynamic global resource pool capable of forming open peer-to-peer AI networks where independently owned devices cooperate through decentralized coordination. Computation is scheduled dynamically instead of requiring every device to take part in every inference request, based on hardware capability, network locality, trust boundaries, and resource availability.

This paper discusses architectural principles, networking strategies, distributed storage, scheduling, communication optimization, deployment models, security considerations, sustainability implications, and open research challenges. The work presents, without prescribing a specific protocol, a systems-level architecture intended to remain applicable across future hardware generations and networking technologies, and describes an early open-source prototype that implements its coordination layers using existing peer-to-peer infrastructure.

1. Introduction

Artificial intelligence is rapidly becoming a fundamental layer of modern computing. Software development, scientific research, healthcare, education, manufacturing, and robotics increasingly rely on machine learning models capable of performing complex reasoning and automation tasks. Yet despite remarkable advances in model capabilities, the infrastructure used to execute these models remains largely centralized: most inference requests are processed inside large data centers operated by a small number of cloud providers, with prompts transmitted to remote servers and responses returned over the network.

This model has enabled rapid adoption of large language models, but it introduces long-term challenges. Organizations become increasingly dependent on external infrastructure for intelligence; every interaction requires connectivity, introduces operational cost, and deepens vendor dependence. As AI usage grows from occasional

requests to continuous integration into software development, enterprise workflows, autonomous systems, and edge devices, the cost of centralized inference scales proportionally with usage, while compute capacity remains geographically concentrated in a limited number of regions.

At the same time, modern computing devices are becoming increasingly capable of executing AI workloads locally. Consumer laptops ship with dedicated neural processing units (NPUs), desktop computers routinely include powerful GPUs, mobile devices contain specialized accelerators, and edge systems integrate domain-specific inference hardware. Much of this capacity sits idle for significant portions of the day. The result is a contradiction: centralized providers continuously expand compute infrastructure to satisfy growing AI demand, while billions of capable devices already exist with substantial unused processing power, memory, storage, and bandwidth.

The question addressed in this paper is therefore not whether larger models can be built, but whether the infrastructure used to execute them can itself become decentralized.

We propose the concept of a Global AI Computer, a peer-to-peer architecture that transforms independently owned devices into a shared AI infrastructure layer. Instead of viewing each computer as an isolated execution environment, participating devices cooperate through distributed storage, capability-aware scheduling, and decentralized networking to collectively provide AI services. Heterogeneity is treated as a fundamental design assumption rather than a limitation: one participant may contribute storage capacity, another may execute transformer layers on a GPU, while a third provides embedding generation through an integrated NPU. The architecture is intentionally independent of any specific machine learning framework, networking protocol, hardware vendor, or model architecture, so that its principles remain applicable as hardware and software continue to evolve.

Beyond computational efficiency, this work is motivated by restoring agency over AI infrastructure. As AI becomes integrated into software, communication, education, and research, dependence on centralized inference services concentrates computational resources, operational costs, and decision-making within a small number of infrastructure providers. The proposed architecture instead treats intelligence as a distributed resource that can be contributed, coordinated, and governed by individuals, communities, and organizations. Improving utilization of hardware that already exists may also reduce the need for continuously expanding dedicated AI infrastructure, although quantifying such sustainability effects remains a subject for future empirical study.

Ultimately, the goal is not to eliminate centralized computing. Decentralized AI infrastructure is presented here as an additional deployment model, one that utilizes existing

resources more efficiently while giving users greater control over where computation occurs, how resources are utilized, and how AI systems evolve over time.

2. Design Goals and Architectural Principles

The objective of the proposed architecture is not simply to distribute computation across multiple devices. Existing distributed systems have demonstrated that computation can already be partitioned across clusters under controlled environments. Instead, the Global AI Computer addresses a broader problem: how independently owned, heterogeneous devices can cooperate to provide a unified AI infrastructure while remaining scalable, resilient, and practical.

To achieve this objective, the proposed architecture is guided by several design principles.

2.1 Decoupling Storage and Computation

Distributed storage and distributed computation are different problems and benefit from being solved separately. Model parameters, checkpoints, embeddings, and similar artifacts should be replicated widely, because replication is what buys availability and fault tolerance. Executing an inference request has the opposite character: it should involve only the devices actually needed to meet a latency and throughput target, since every additional participant adds coordination cost. Keeping the two concerns apart lets storage optimize for durability while computation optimizes for execution efficiency, and lets either evolve without disturbing the other.

2.2 Capability-Aware Scheduling

Devices differ in processing power, memory capacity, storage performance, network bandwidth, and specialized AI hardware, and these differences are large enough that uniform work assignment wastes most of the available capacity. The architecture therefore assumes nodes contribute according to what they can do rather than performing identical work: a workstation with a high-performance GPU may execute computationally intensive inference, while a lightweight edge device contributes storage, embeddings, routing, or smaller workloads. Because AI hardware continues to change, the scheduler reasons about abstract capabilities (available memory, compute throughput, accelerator availability, network characteristics) rather than specific processor architectures, so new hardware becomes usable without scheduler changes.

2.3 Communication Minimization

Communication overhead can dominate execution time in distributed inference, and the intuition that every participating device should contribute to every request makes this worse rather than better: it maximizes synchronization and end-to-end latency simultaneously. Communication is better treated as a scarce resource. The scheduler should therefore build execution plans that minimize inter-device traffic while maximizing useful computation, keeping work local to devices that already

hold the required model components. Beyond latency, this reduces bandwidth consumption and improves resilience when network conditions are poor, since a device that talks to fewer peers has fewer ways to fail.

2.4 Dynamic Resource Allocation

Participation in a peer-to-peer network changes continuously. Devices join and leave without warning, disconnect temporarily, become resource constrained, or change availability over the course of a day. Treating this as an exceptional condition would make the common case a failure case, so the architecture instead adapts continuously: workloads are reallocated, replicated data is redistributed, and alternative execution paths are selected as conditions change. Graceful degradation is a primary design objective rather than a recovery procedure.

2.5 Heterogeneous Execution

The architecture makes no assumptions about execution environments. Workloads may run on GPUs, NPUs, CPUs, dedicated accelerators, browser-based runtimes, embedded processors, or hardware that does not yet exist. What the network coordinates is computational capability, not software implementation, which allows execution engines to evolve independently while remaining compatible with the surrounding infrastructure.

2.6 Deployment Independence

AI infrastructure operates under widely varying ownership and privacy constraints, from individual devices and community networks to organizational infrastructure and open global resource pools. The architecture supports private, public, and hybrid deployments (detailed in Section 7) over the same scheduling and storage layers, so a deployment can move between these models without architectural change.

2.7 Design Summary

Taken together, these principles describe an architecture optimized for efficient coordination between distributed resources rather than for maximum distribution. Storage is treated independently from computation, execution adapts to heterogeneous hardware, communication is minimized wherever possible, and scheduling responds continuously to changing network conditions. The sections that follow build the architecture on this foundation.

3. The Global AI Computer Architecture

The proposed architecture models artificial intelligence infrastructure as a collection of cooperating resources rather than isolated computers or centralized servers. Instead of assuming that every participating device performs the same function, the architecture assigns responsibilities according to hardware capability, resource availability, and network conditions.

At a high level, the Global AI Computer consists of four logical layers:

- Storage Layer
- Compute Layer
- Scheduling Layer
- Communication Layer

Each layer is responsible for a distinct function and can evolve independently without requiring changes to the overall system architecture.

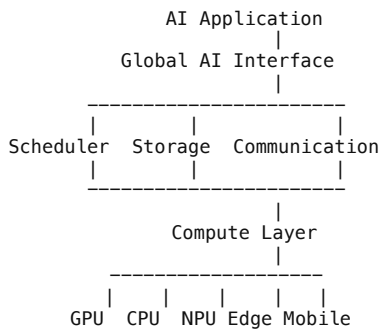


Fig. 1: High-level architecture of the Global AI Computer.

Separating these responsibilities allows the system to optimize storage, computation, and networking independently while avoiding unnecessary coupling between components.

3.1 Storage Layer

The Storage Layer maintains model parameters, checkpoints, embeddings, metadata, and execution artifacts. Where conventional systems give every execution node a complete copy of every model, this architecture replicates data according to availability, demand, and the storage capacity peers actually contribute. Replication buys fault tolerance, high availability, efficient synchronization, and shorter download times for devices joining the network.

Participation in storage does not imply participation in computation. A device may contribute storage without ever executing inference, and a device executing inference may store only the subset of model data its assigned workload requires. Because the two roles are separate, storage policy can be tuned for durability without constraining how execution is scheduled.

3.2 Compute Layer

The Compute Layer executes AI workloads across devices that differ substantially in capability: desktop GPUs, CPUs, neural processing units, dedicated inference accelerators, embedded AI hardware, and processors that have not yet shipped. Traditional clusters treat such variation as a problem to be normalized away; here it is the operating assumption. A high-memory server may execute large inference workloads while a mobile device performs lightweight inference, embedding generation, or preprocessing, so computational responsibility follows capability rather than device category.

3.3 Scheduling Layer

The Scheduling Layer coordinates the system. Its job is not simply to assign work but to decide how available resources

should cooperate to satisfy a particular request, weighing compute capacity, accelerator availability, memory, current workload, network latency, storage locality, trust boundaries, and device availability at the same time.

Instead of a fixed execution pipeline, the scheduler builds an execution graph for each request. As devices join, leave, or change state, later requests may produce entirely different graphs without any visible change in application behavior. This is what allows one architecture to span environments from a small private cluster to a globally distributed peer-to-peer network.

3.4 Communication Layer

The Communication Layer handles peer discovery, capability advertisement, task coordination, data synchronization, execution messaging, and fault detection. It prescribes no particular networking protocol, assuming only that devices communicate through secure peer-to-peer mechanisms supporting decentralized discovery, authenticated communication, and efficient data transfer. Keeping communication separate from scheduling means transport protocols can improve without changes to execution logic.

3.5 Architectural Separation

Storage, computation, scheduling, and communication solve different problems, yet many distributed AI systems combine them into a single implementation, which makes each harder to optimize as deployments grow. Separating them into independent layers is a central position of this work.

Layer	Primary Responsibility
Storage	Replication and availability of AI artifacts
Compute	Execution of inference workloads
Scheduling	Selection and coordination of participating devices
Communication	Peer discovery and information exchange

This separation improves modularity while allowing each layer to evolve independently as hardware capabilities, networking technologies, and AI models continue to advance.

The following section builds upon this architecture by describing how inference requests are executed across the network while minimizing communication overhead and adapting to heterogeneous hardware.

4. Execution Model

The previous section described the logical components of the Global AI Computer; this section explains how they cooperate during an inference request. No fixed cluster is assumed and no requirement exists that every available device take part in every computation. Instead, an execution plan is constructed per request from the current state of the network, through three progressively refined views: the **Resource Graph**, the **Capability Graph**, and the **Execution Graph**. Together these reduce a large, continuously changing network to a small set of cooperating resources chosen for one specific request.

4.1 Resource Graph

The Resource Graph represents every node currently connected to the Global AI Computer, where each node is a device contributing at least one of computation, storage, memory, or networking. It carries no information about hardware capability or current workload; its only purpose is to record what infrastructure exists.

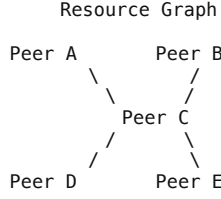


Fig. 2: The Resource Graph representing participating peers.

The graph evolves continuously as devices join and leave.

4.2 Capability Graph

Where the Resource Graph records which resources exist, the Capability Graph records what they can currently provide. Each device periodically advertises information relevant to scheduling:

- Available compute capacity
- Memory availability
- Accelerator capabilities
- Storage capacity
- Current utilization
- Estimated network latency
- Administrative or trust boundaries

The Capability Graph is dynamic where the Resource Graph is structural: it reflects the operational state of the network at the moment a request arrives.

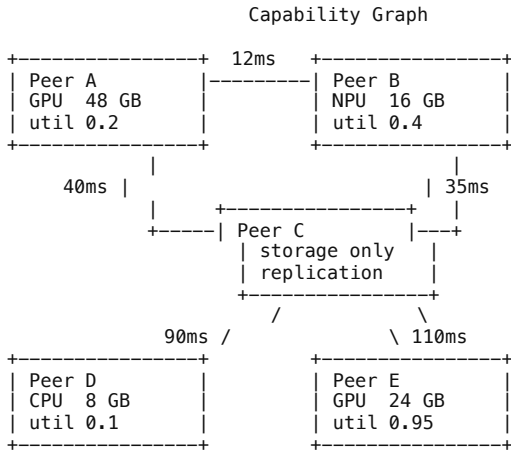


Fig. 3: The Capability Graph. Vertices carry advertised capability and current utilization; edges carry measured latency. Peer C contributes storage only, and Peer E is capable but saturated.

Because capability is expressed in these abstract terms, the scheduler reasons about resources without depending on

hardware vendors, operating systems, or execution frameworks.

4.3 Execution Graph

The Execution Graph is the subset of resources selected for one inference request. Unlike the other two graphs it is temporary: it exists for the lifetime of the request and is discarded when execution completes.

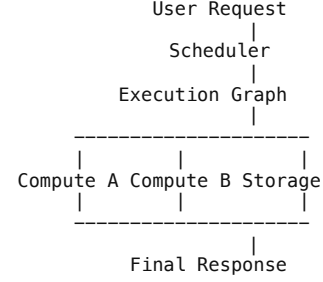


Fig. 4: The Execution Graph constructed for a single inference request.

Each node is a participating computational resource and each edge is a flow of information required during execution. Later requests may produce entirely different graphs as hardware availability, network conditions, scheduling policy, or workload characteristics change.

4.4 Request Lifecycle

Every inference request follows the same six-stage process.

1. Request Submission. An application submits a request through a standard interface. The request carries no assumptions about where computation will occur.

2. Resource Discovery. The scheduler reads the current Resource Graph to determine which devices are participating. Devices joining or leaving modify this graph without any effect on application behavior.

3. Capability Evaluation. The scheduler derives a Capability Graph from the current state of those resources, covering computational performance, memory availability, accelerator support, storage locality, network latency, workload, and trust policy. Only resources relevant to the request are considered.

4. Execution Graph Construction. Using the Capability Graph, the scheduler constructs an Execution Graph for the request.

The objective is not to maximize the number of participating devices but to minimize overall execution cost while holding latency, reliability, and utilization within acceptable bounds. Section 4.5 states this objective formally. Its practical consequence is that many online devices will not participate in a given request.

5. Distributed Execution. Participating devices execute only the workloads assigned to them. No specific inference strategy is prescribed, so execution engines may partition work differently while remaining compatible with the same architectural model.

6. Response Assembly. Intermediate results are assembled into the final response and returned to the requesting application, and the Execution Graph is discarded. Each subsequent request repeats the process independently against the network as it then stands.

4.5 Scheduler Optimization

The Scheduling Layer constructs an Execution Graph by selecting an appropriate subset of participating peers while minimizing overall execution cost and satisfying resource constraints. It does not attempt to utilize every available device; it identifies the smallest collection of resources capable of executing a given inference request efficiently.

Let

- N denote the set of available peers,
- $x_i \in \{0, 1\}$ indicate whether peer i participates in the Execution Graph,
- c_i represent the computational capability of peer i ,
- m_i denote available memory,
- l_{ij} denote communication latency between peers i and j ,
- r_i denote the estimated reliability of peer i , and
- t_i denote the trust score associated with peer i .

The scheduler seeks to minimize the overall execution cost

$$\min \left(\sum_i x_i C_{\text{compute}}(i) + \lambda \sum_{i,j} x_i x_j C_{\text{comm}}(i, j) - \alpha \sum_i x_i r_i - \beta \sum_i x_i t_i \right)$$

subject to the constraints

$$\sum_i x_i c_i \geq C_{\text{required}}$$

$$\sum_i x_i m_i \geq M_{\text{required}}$$

$$\text{Latency}(E) \leq L_{\text{max}}$$

$$\text{Trust}(E) \geq T_{\text{min}}$$

where

- $C_{\text{compute}}(i)$ represents the computational cost of running work on peer i ,
- $C_{\text{comm}}(i, j)$ represents the communication overhead between peers i and j , estimated from l_{ij} ,
- L_{max} denotes the maximum acceptable latency, and
- T_{min} represents the minimum trust threshold required by the workload.

The weights λ , α , and β set how aggressively the scheduler trades communication cost against reliability and trust. They are deployment parameters rather than constants: a private cluster on a local network can afford a small λ and a small β , whereas an open network spanning unreliable links needs both to be large.

The proposed architecture intentionally does not prescribe a particular optimization algorithm. Future implementations may employ graph optimization techniques, heuristic scheduling, integer programming, reinforcement learning, or evolutionary search while preserving the same architectural

objective of minimizing execution cost under dynamic network conditions.

Algorithm 1: Execution Graph Construction

Input:
Resource Graph G
Capability Graph C
Inference Request R

Output:
Execution Graph E

1. Discover active peers
2. Retrieve current capabilities
3. Filter peers satisfying resource constraints
4. Rank peers according to
 - compute capability
 - available memory
 - network latency
 - storage locality
 - trust score
 - current utilization
5. Estimate communication cost
6. Construct candidate Execution Graph
7. Remove redundant participants
8. Verify scheduling constraints
9. Dispatch execution
10. Monitor execution and reconstruct graph if failures occur

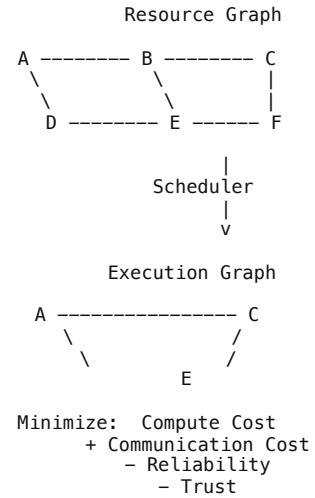


Fig. 5: The scheduler reduces the Resource Graph to the smallest Execution Graph that satisfies the request.

4.6 Locality-Aware Scheduling

Communication often becomes the dominant cost in distributed inference, so the scheduler attempts to preserve computational locality whenever possible. Execution plans prefer resources that already possess the required model components, exhibit lower communication latency, or belong to the same administrative domain. Reducing communication improves latency while minimizing unnecessary movement of intermediate data across the network.

4.7 Dynamic Adaptation

Unlike traditional computing clusters, peer-to-peer environments change continuously: devices disconnect unexpectedly, become temporarily overloaded, gain

additional resources, or rejoin after periods of inactivity. Treating these events as failures would make the common case exceptional, so the architecture instead assumes continuous infrastructure evolution and schedules each inference request independently against the current Capability Graph, allowing the system to adapt as conditions change.

4.8 Why Not Use Every Device?

It may seem natural to assume that adding devices to a computation always improves performance. Each additional participant also adds communication, synchronization, and coordination overhead, and beyond some point these costs outweigh the computational benefit of the extra hardware. The effect is compounded by tail latency: when a request must wait on every participant, the slowest one determines completion time, and the probability that at least one participant is slow grows with the size of the graph [8]. Adding devices therefore increases both the mean coordination cost and the variance of the response time.

The architecture consequently optimizes for efficient cooperation rather than maximum participation. The target is the smallest Execution Graph capable of completing the workload, not the largest one the network can assemble.

5. Distributed Model Storage

Unlike traditional AI deployments where complete models are stored on centralized infrastructure or replicated entirely across every execution node, the proposed architecture treats AI models as distributed network resources.

Model parameters, metadata, tokenizers, embeddings, and related artifacts are not files owned by individual machines; they exist collectively within the storage layer of the Global AI Computer.

Participating devices contribute storage according to available capacity while the network maintains sufficient replication to ensure availability and resilience.

Separating storage from computation enables each layer to evolve independently. A device may contribute storage without participating in inference, while another device may execute inference using model components obtained from nearby peers.

5.1 Model Chunking

Requiring every participating device to store complete model parameters would limit participation to devices large enough to hold the entire model, which is precisely the constraint the architecture exists to remove. Model artifacts are therefore divided into independently addressable chunks, each a logical portion that can be stored, replicated, verified, and transferred on its own.

No particular partitioning scheme is fixed. Implementations may organize model data by transformer layer [10], by tensor, by parameter block, by quantized representation, or by whatever layout suits the execution engine underneath. What matters to the storage layer is only that model artifacts appear as collections of independently managed objects.

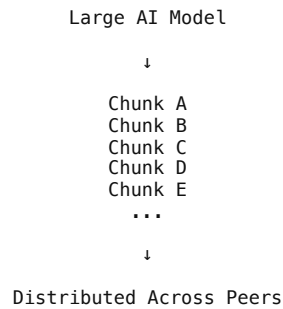


Fig. 6: Model chunking and distribution across peers.

This abstraction enables efficient replication while reducing the amount of data transferred when devices join the network.

5.2 Replication

Availability in a peer-to-peer system comes from redundancy rather than from any single reliable machine, so each model chunk may exist on several participating devices. Replication improves fault tolerance, reduces download latency, raises availability, distributes artifacts geographically, and speeds recovery after a peer fails.

Replication policy is deliberately independent of execution policy. Frequently accessed components can be replicated more aggressively than rarely used ones, letting storage strategy follow observed workload without any corresponding change to how requests are scheduled.

5.3 Discovery

Storage is only useful if artifacts can be found, so the storage layer maintains a distributed mechanism for discovering where model components are currently available. When a request is scheduled, the scheduler consults this metadata to identify peers that already hold the required artifacts and prefers them when constructing the Execution Graph. Storage discovery is thus an input to scheduling rather than a separate stage that runs before it, which is what makes the locality preference of Section 4.6 possible in the first place.

5.4 Verification

In decentralized environments, every model component should be independently verifiable regardless of where it is obtained. Trust in the peer that served an artifact is therefore unnecessary: stored objects are validated using cryptographic integrity mechanisms appropriate for the deployment environment, enabling any participant to confirm that retrieved model components are authentic before incorporating them into inference execution. Verification thus establishes confidence in stored data without requiring centralized storage providers.

5.5 Caching

Inference workloads frequently exhibit temporal locality: popular models, frequently executed layers, embeddings, and intermediate artifacts are reused across many requests. Participating devices therefore maintain local caches of

recently accessed model components, reducing repeated network transfers and allowing future Execution Graphs to preferentially utilize resources that already possess the required data. As demand shifts, cached artifacts evolve according to network usage without centralized coordination.

By combining chunking, replication, discovery, verification, and adaptive caching, the storage layer transforms AI models from static files into distributed network resources capable of supporting dynamic execution across heterogeneous infrastructure.

5.6 Content-Addressable Model Storage

Model artifacts may be identified by a cryptographic content hash rather than by physical storage location. This enables immutable storage, efficient deduplication, integrity verification, and deterministic retrieval throughout the peer-to-peer network.

Content-addressable storage also allows identical model components shared across different model versions to be reused without redundant replication. Since storage references remain independent of physical location, peers may retrieve artifacts from the nearest available replica while maintaining cryptographic verification of correctness.

By combining content-addressable storage with replication and adaptive caching, the Global AI Computer can reduce storage redundancy, minimize network traffic, and improve model availability without introducing centralized storage services.

6. Peer-to-Peer Communication Model

Communication provides the coordination mechanism that enables independently owned devices to operate as a unified AI infrastructure.

Unlike conventional client-server architectures where all requests pass through centralized infrastructure, the proposed architecture assumes decentralized communication between participating peers.

The communication layer handles discovery, capability exchange, execution coordination, synchronization, and fault detection, and it does so without committing to a particular networking protocol so that transport technology can improve independently of storage, scheduling, and execution.

6.1 Peer Discovery

The layer's first responsibility is finding participating resources. As devices join and leave, peers exchange enough information to maintain a current view of available infrastructure, and the resulting Resource Graph gives the scheduler its picture of the network without assuming anything about topology or deployment scale. Discovery runs continuously rather than as a startup phase, so infrastructure changes flow naturally into subsequent scheduling decisions.

6.2 Capability Advertisement

Finding a peer says nothing about what it can do, so devices also advertise their current state: available computational resources, memory capacity, accelerator availability, storage contribution, current utilization, estimated communication latency, and any administrative policies that constrain what they will accept. Because all of these change over time, advertisements are refreshed periodically. The Capability Graph therefore reflects what the network can do now rather than what its hardware could do in principle, a distinction that matters most for peers that are capable but saturated.

6.3 Task Coordination

Once an Execution Graph exists, participating devices coordinate only with the peers in that graph. Execution state is never broadcast across the network. Because peers do not share a global clock, coordination relies on the causal ordering established by the messages themselves [1] rather than on synchronized timestamps.

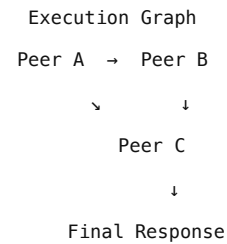


Fig. 7: Task coordination within a temporary execution graph.

Confining communication to participating resources reduces bandwidth consumption and synchronization overhead together, which is what allows the network to grow without requiring every peer to take part in every request.

6.4 Communication Complexity

Because communication is limited to the temporary Execution Graph built for each request, the network functions as a global resource pool rather than a global execution graph.

Let N be the number of devices connected to the Global AI Computer and k the number participating in a particular request, with $k \ll N$. Communication complexity then depends on the size of the Execution Graph rather than on the size of the network. A linear execution pipeline needs $O(k)$ messages, and more general graph-based execution needs approximately $O(k \log k)$ under hierarchical coordination. Broadcast-based approaches, by contrast, require $O(N)$ or $O(N^2)$ messages depending on the synchronization strategy, and these costs grow with the network whether or not the additional peers contribute anything to the request.

Scaling therefore comes from selective participation rather than global synchronization, which is what allows the network to grow substantially without a proportional increase in per-request communication.

6.5 Direct Peer Communication

Communication should occur directly between participating peers wherever the network permits it, since direct paths reduce latency, remove intermediaries, and make transfer of execution data more efficient. No particular transport is required: implementations may use different peer-to-peer protocols according to deployment needs, because the communication layer defines logical responsibilities rather than implementation details.

6.6 Adaptive Communication

Latency, bandwidth, peer availability, and routing characteristics all change during system operation. Communication paths are therefore established dynamically for each Execution Graph rather than remaining permanently fixed, and successive inference requests may use entirely different paths while preserving identical application behavior. This adaptability enables the architecture to operate across local organizational deployments, geographically distributed infrastructures, and large public peer-to-peer networks.

7. Deployment Models

Organizations operate under different privacy requirements, regulatory constraints, ownership models, and performance objectives, so the Global AI Computer supports several deployment modes without changes to the underlying architecture. Enterprise and public infrastructure are not treated as separate systems but as different operational configurations of the same principles, differing mainly in how the trust term of Section 4.5 is populated.

7.1 Private Deployment

In a private deployment, all participating resources remain under the control of a single organization or trusted administrative domain.



Fig. 8: Private deployment model.

Such deployments prioritize data privacy, low network latency, regulatory compliance, administrative control, and predictable resource availability. Because every device belongs to the same organization, scheduling can assume high trust, since a large β buys nothing when every t_i is 1, and can keep communication inside organizational boundaries. Private deployment is expected to be the

primary adoption model for enterprises, research institutions, and anyone operating sensitive workloads.

7.2 Public Deployment

Public deployments allow independently owned devices to cooperate within an open peer-to-peer network.

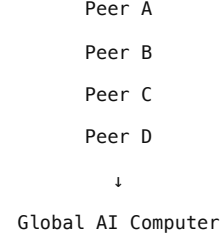


Fig. 9: Public deployment model.

Open participation raises problems that private deployment does not: establishing identity, accumulating reputation, verifying that advertised resources exist, and giving peers a reason to contribute at all. Decentralized incentive design has been studied extensively outside AI infrastructure, in peer-to-peer file distribution [4] and in permissionless consensus [13], and the mechanisms developed there apply here without modification. The architecture keeps these concerns outside the core execution model precisely so that trust, reputation, and incentive mechanisms can evolve independently of how requests are scheduled.

7.3 Hybrid Deployment

Most practical deployments will combine private infrastructure with public resources.

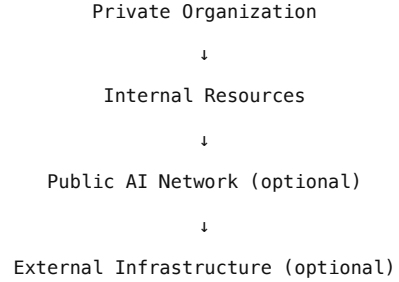


Fig. 10: Hybrid deployment model.

Such deployments let an organization prioritize internal resources while drawing on external capacity when internal resources are exhausted, which permits gradual adoption without migrating away from existing infrastructure. In scheduling terms, a hybrid deployment is one where the trust threshold $T_{\min}$ varies by workload: sensitive requests are confined to internal peers, while others may leave the organizational boundary.

8. Fault Tolerance and Resilience

Devices may disconnect unexpectedly, experience temporary network failures, become overloaded, or leave the

network permanently. The scheduler evaluates resource availability independently for each inference request, replication within the Storage Layer preserves data availability, and dynamic construction of Execution Graphs allows subsequent requests to route around unavailable participants. As the network grows, resilience emerges through redundancy rather than dependence on individual machines, and graceful degradation becomes a normal operating characteristic rather than a recovery procedure.

9. Security and Trust

Distributed AI infrastructure introduces security challenges beyond those encountered within centralized deployments. The architecture distinguishes between infrastructure trust and computational capability: a device may possess sufficient computational resources while remaining unsuitable for sensitive workloads due to administrative or policy constraints. Trust is therefore an input to scheduling rather than a property of networking.

Trust may be established through organizational identity, administrative policy, cryptographic authentication, reputation systems, or secure execution environments. The architecture commits to none of these. A private deployment may rely entirely on organizational identity, while an open network needs mechanisms designed for participants who may be actively adversarial rather than merely unreliable, the setting addressed by Byzantine fault-tolerant protocols [2]. Verifying that a peer returned the output it was asked to compute, rather than a cheaply fabricated substitute, remains an open problem for public deployment and is discussed in Section 13.

10. Prototype Implementation

To examine the practicality of the proposed architecture, we developed *ai-mesh*, an open-source software development kit that implements the coordination and scheduling layers described in this paper, together with an integration into Peersky [20], a peer-to-peer web browser developed by the author. The prototype does not implement every component of the architecture. Its purpose is narrower: to establish that decentralized peer discovery, capability advertisement, capability-aware scheduling, dynamic layer assignment, and pipelined activation streaming can be built today from existing peer-to-peer infrastructure, independently of any particular inference backend.

10.1 System Components

The prototype consists of five modules whose responsibilities correspond directly to the abstractions introduced in Sections 3 and 4.

Module	Architectural Role
Peer manager	Resource Graph maintenance: discovery, connection lifecycle, liveness
Capability advertisement	Capability Graph construction
Scheduler	Execution Graph construction (Algorithm 1) and cost minimization
Layer manager	Layer assignment for pipeline-parallel execution
Activation router	Task coordination and streaming of intermediate activations

Peer discovery uses Hyperswarm [19], a distributed hash table-based networking stack. Peers join a swarm identified by the hash of a shared topic string and establish end-to-end encrypted connections without centralized coordination. Upon connection, each peer immediately transmits an announcement advertising its capabilities, including available memory, accelerator availability, and current utilization. Liveness is maintained through periodic heartbeats, and round-trip latency and delivery reliability are measured continuously through pings; peers that remain silent beyond a timeout are pruned from the Resource Graph, which triggers reassignment of their responsibilities.

The scheduler is a direct implementation of Algorithm 1. Given the current Capability Graph and a request's constraints, it filters candidates on latency and trust, ranks them by a composite of compute score, utilization, measured latency, reliability, and trust, then greedily grows the smallest Execution Graph that satisfies aggregate compute and memory requirements and prunes redundant participants. The objective function of Section 4.5 is implemented with configurable weights: the communication penalty, reliability reward, and trust reward are runtime parameters rather than fixed constants, and trust values are supplied by a pluggable provider so that private deployments can assign full trust while public deployments substitute their own mechanisms. The scheduler supports three execution modes: *delegate*, which places an entire request on the single best executor; *pipeline*, which partitions layers across peers and streams activations stage to stage; and *auto*, which prefers a single capable executor and falls back to the pipeline, reflecting the locality principle of Section 4.6.

Messages use a compact binary framing consisting of a one-byte type, a length-prefixed JSON control payload, and an optional raw binary payload. This allows control metadata and tensor activations to travel in a single frame without encoding overhead, which matters when intermediate activations are streamed repeatedly through a pipeline.

The layer manager partitions a model's transformer layers into contiguous ranges assigned proportionally to peer capacity, and ordering those assignments by layer index yields the pipeline that instantiates an Execution Graph for multi-peer execution. Capacity is computed from the bytes a layer actually occupies at the model's quantization, not from the layer count, because the two diverge sharply at scale: a 16 GB peer holds roughly a quarter of a 32-layer 8B model but barely one layer of a 126-layer 405B model. Estimating from depth alone would credit that peer with a share it could never load, and the resulting assignment would report complete coverage for a pipeline that cannot execute. Unified-memory systems are treated as a single pool rather than as separate RAM and VRAM budgets for the same reason. When capacity across the mesh is insufficient, coverage remains partial and pipeline requests are refused rather than being forced onto peers that lack the memory to serve them. When a peer departs, its range is redistributed across those that remain.

The activation router forwards intermediate activations from each stage to the next, with per-request timeouts, cancellation, and completion signaling. Because the swarm

does not guarantee that the terminal stage shares a connection with the peer that submitted the request, results and errors carry the request origin and are relayed back along the pipeline when no direct path exists. The execution backend is pluggable, consistent with the position taken throughout this paper that coordination should not be coupled to a particular inference strategy: any backend implementing a small interface can participate. Backends declare whether they can evaluate a strict subset of a model's layers, and those that cannot are excluded from pipeline mode. This distinction is load-bearing. A backend that satisfied the interface by returning activations unchanged would produce runs that appear to traverse every stage successfully while no stage computed anything, so the prototype treats the absence of partial execution as an error rather than as a degraded mode. The prototype provides a llama.cpp [18] backend, which executes complete models and is therefore available for delegation but not for pipeline stages.

Within Peersky, the SDK runs in the browser's main process and is controlled entirely from user-facing settings, which expose the execution mode, swarm topic, hardware capability overrides, and every scheduler weight and constraint. A participating browser can both submit requests to the mesh and serve requests scheduled onto it by other peers through its local inference engine, and the built-in chat interface transparently routes through the mesh scheduler when enabled, falling back to local execution if no remote executor is available. Model artifacts are currently retrieved over HTTPS, with content-addressable distribution across peers (Section 5.6) as the intended replacement.

10.2 Current Status and Limitations

We report the status of the prototype candidly, since the gap between what it coordinates and what it computes is the most important thing to be clear about.

The coordination and scheduling stack is validated by automated tests running over real Hyperswarm connections on a local DHT. Peers discover one another and exchange capabilities over encrypted streams, the scheduler constructs Execution Graphs under latency, trust, and resource constraints, layer assignments rebalance when peers join or depart, and requests execute in all three modes, including delegated chat answered by a remote peer and pipeline requests whose activations traverse every stage in order. Deterministic backends are used for the pipeline tests so that stage ordering is verifiable, while delegated full-model inference through the llama.cpp backend exercises real model execution.

What the prototype does not do is execute one large model across several peers. Doing so requires a backend that evaluates an assigned layer range and exports the intermediate activations, and llama.cpp does not currently expose the tensor access this needs. The consequence is worth stating plainly rather than describing the pipeline as merely incomplete: with the available backends, the pipeline path validates coordination and carries no computation, and delegate mode succeeds only when some single peer can already run the model unaided. The prototype therefore refuses pipeline mode outright when no participating

backend supports partial execution, and refuses it when byte-aware capacity estimates show the mesh cannot hold the model, in preference to producing a run that appears to have succeeded.

Two further gaps remain. We have not performed wide-area measurements across independently owned devices, so the communication costs modelled in Section 6.4 are analytic rather than observed. Mid-inference fault recovery is also unimplemented: a peer that fails during a request causes that request to fail, and only subsequent requests are routed around it.

These limitations follow from the central design position of this paper. The coordination architecture is separable from the execution strategy, and it is the coordination and scheduling layers that the prototype implements first. That separation is also what makes the remaining work tractable: a backend supporting layer-range execution can be added behind the existing interface without changes to discovery, scheduling, or routing.

11. Related Work

The idea of distributing computation across multiple machines has been explored extensively within distributed systems research. High-performance computing clusters, volunteer computing platforms, distributed storage systems, and modern distributed machine learning frameworks have each addressed different aspects of large-scale computation. The proposed Global AI Computer builds upon these ideas while targeting a different systems objective: decentralized AI infrastructure operating across heterogeneous, independently owned devices.

Distributed computing platforms such as MapReduce [7] and parameter server architectures [11] have demonstrated that computational workloads can be partitioned across multiple machines to improve throughput and reliability. These systems typically assume relatively stable infrastructure, homogeneous hardware, and centralized scheduling. In contrast, the proposed architecture assumes continuously changing participation and heterogeneous hardware capabilities as fundamental system properties rather than operational exceptions.

Volunteer and grid computing projects such as BOINC [6] and computational grids [5] have shown that independently owned computers can collectively solve computationally intensive problems. Such systems generally focus on embarrassingly parallel workloads where tasks can execute independently with minimal communication. AI inference introduces substantially different requirements, including low-latency execution, model availability, coordination of intermediate computation, and adaptive scheduling based on hardware capabilities.

Peer-to-peer storage and lookup systems such as Chord [3] and BitTorrent [4] have addressed reliable replication, fault tolerance, and decentralized data availability. These ideas directly motivate the separation of storage and computation proposed in this work. Decoupling model storage from inference execution allows storage policies and execution strategies to evolve independently.

Recent distributed machine learning frameworks [9], [11], [12] primarily focus on training large models or executing inference within controlled data center environments. These systems typically assume high-bandwidth interconnects, managed infrastructure, and relatively predictable hardware configurations. Closest to this work, Petals [16] and Hivemind [17] execute large transformer models across volunteer devices by partitioning layers among peers, demonstrating that decentralized inference is feasible in practice. However, these systems commit to a specific model family and execution strategy. The proposed architecture instead targets open, heterogeneous environments where partitioning and execution remain pluggable concerns beneath capability-aware scheduling, and where communication cost becomes an explicit scheduling consideration.

Edge computing [14] and federated learning [15] have similarly emphasized executing computation closer to end users by leveraging local hardware. The Global AI Computer extends this idea beyond individual devices by enabling groups of heterogeneous edge resources to cooperate through decentralized scheduling and communication while preserving locality whenever possible.

Although these research areas address complementary problems, none individually define a generalized architectural framework for decentralized AI infrastructure spanning storage, computation, scheduling, communication, and heterogeneous resource coordination across both private and public deployments.

This architecture does not replace those systems. It is better read as a unifying systems abstraction that integrates concepts from distributed computing, peer-to-peer networking, distributed storage, and modern AI infrastructure into a common architectural model.

12. Architectural Comparison

Property	Centralized Cloud AI	Traditional HPC Cluster	Distributed AI Frameworks	Proposed Global AI Computer
Infrastructure Ownership	Single organization	Single organization	Usually single organization	Public, Private, or Hybrid
Resource Discovery	Static	Static	Limited	Dynamic Peer Discovery
Hardware Assumption	Mostly homogeneous	Homogeneous	Partially heterogeneous	Fully heterogeneous
Storage	Centralized	Shared	Framework-dependent	Distributed
Scheduling	Centralized	Cluster scheduler	Framework scheduler	Capability-aware scheduler
Fault Tolerance	Infrastructure redundancy	Cluster redundancy	Limited	Native peer resilience
Scalability	Provider expansion	Hardware expansion	Cluster scaling	Network growth
Communication	Centralized	High-speed local network	Cluster network	Adaptive peer-to-peer
Deployment Model	Cloud	Private	Private	Public, Private, Hybrid
Hardware Evolution	Vendor dependent	Administrator managed	Framework dependent	Hardware agnostic

13. Discussion

The Global AI Computer describes a shift from centralized AI services toward distributed AI infrastructure, treating the computing devices that already exist as potential participants in a shared computational ecosystem rather than building ever-larger centralized facilities. Several research challenges remain open, and the prototype of Section 10 makes some of them concrete.

13.1 Scheduling

Efficient scheduling across heterogeneous hardware remains the central research problem. The objective of Section 4.5 is a static estimate: it uses advertised capability and measured latency, but nothing about how a peer has actually performed on comparable work. Future schedulers may incorporate historical throughput, observed communication cost, workload characteristics, and energy efficiency, and may learn weights from execution outcomes rather than accepting them as configuration.

13.2 Model Partitioning

Different model architectures expose very different partitioning structures, and the choice interacts strongly with network topology. Pipeline partitioning of a dense transformer requires every token to traverse every stage in sequence, so per-token latency accumulates across hops and the approach suits low-latency links far better than wide-area ones. Sparse mixture-of-experts models behave differently: each token activates a small fraction of the parameters, and experts are independent, immutable objects that map naturally onto the content-addressable storage of Section 5.6. This suggests that for such models the more valuable contribution of a peer-to-peer mesh may be distributing the parameters rather than distributing the arithmetic, since it allows a group of peers to collectively hold a model that no single participant could store. Determining which partitioning strategy suits which combination of model structure and network conditions remains open, and the architecture deliberately leaves the choice to the implementation.

13.3 Combining Multiple Models

A further question is whether independently hosted models can cooperate within a single inference request. Instead of merging parameters, an execution graph could coordinate specialized models, one performing planning, another retrieval, another reasoning, as distinct nodes with data flowing between them. This form of collaborative inference is a promising direction, though it lies outside the architecture proposed here.

13.4 Verifiable Execution

Public deployment raises a problem private deployment does not: a peer that returns a plausible but fabricated result is far cheaper to operate than one that performs the computation honestly. Reputation limits the damage over time but does not detect a single incorrect response. Redundant execution across independently selected peers, cryptographic attestation from secure execution environments, and probabilistic verification of sampled

results are all plausible directions, and each carries a cost that the scheduling objective would need to represent explicitly.

13.5 Hardware Evolution

AI hardware continues to diversify: GPUs, NPUs, domain-specific accelerators, and embedded inference processors already coexist, and more will follow. Because the architecture schedules against abstract capability rather than any particular processor architecture, new accelerators become additional resources within the Global AI Computer without redesign. This is the practical payoff of treating heterogeneity as an assumption rather than a complication.

13.6 Sustainability

If AI becomes a persistent component of computing infrastructure, the utilization of hardware that already exists becomes an infrastructure question rather than only an efficiency one. Improving that utilization may reduce duplication of dedicated infrastructure while widening access to AI capabilities. Quantifying the environmental effect requires empirical evaluation this paper does not attempt, and any such study would need to account for the energy cost of the coordination and data transfer the architecture introduces, not only the capacity it recovers.

14. Conclusion

This paper introduced the Global AI Computer, a peer-to-peer architecture for distributed artificial intelligence infrastructure. Where conventional approaches concentrate computation in dedicated cloud environments, this architecture separates storage, computation, scheduling, and communication into independent layers coordinated through decentralized networking. Its principal contribution is capability-aware execution through dynamically constructed Execution Graphs, which lets heterogeneous devices cooperate without requiring every participant to execute every request. Because it supports private, public, and hybrid deployment over the same machinery, and commits to no particular networking protocol, hardware architecture, or machine learning framework, it should remain applicable as those layers continue to change.

An early prototype shows that the coordination layers (peer discovery, capability advertisement, capability-aware scheduling, layer assignment, and activation streaming) can be built today on existing peer-to-peer infrastructure. It also shows where the difficulty lies. Coordination proved tractable; distributed execution of a single large model did not, because the inference backends available today evaluate complete models rather than layer ranges. Communication optimization, trust and verification models, partial execution, and efficient use of emerging accelerators all remain open, and settling them will require experimentation and measurement across real deployments rather than further architectural specification.

The broader argument is that increasingly capable AI systems need not become increasingly centralized. Hardware capable of running substantial models already exists in enormous quantity and sits idle for much of the day; whether it can be organized into usable infrastructure is

an engineering question, not a question about the models themselves. If it can, the result would improve utilization of hardware that has already been manufactured, reduce dependence on continuously expanding centralized capacity, widen access, and give individuals and organizations more control over where their computation happens. None of that replaces cloud infrastructure. It adds an architecture alongside it, and expands the ways in which artificial intelligence can be shared, executed, and owned.

15. References

- [1] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [2] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," *Proceedings of OSDI*, 1999.
- [3] I. Stoica et al., "Chord: A Scalable Peer-to-Peer Lookup Service for Internet Applications," *Proceedings of SIGCOMM*, 2001.
- [4] B. Cohen, "Incentives Build Robustness in BitTorrent," *Proceedings of the Workshop on Economics of Peer-to-Peer Systems*, 2003.
- [5] I. Foster and C. Kesselman, *The Grid: Blueprint for a New Computing Infrastructure*. Morgan Kaufmann, 1999.
- [6] D. P. Anderson, "BOINC: A System for Public-Resource Computing and Storage," *Proceedings of the Fifth IEEE/ACM International Workshop on Grid Computing*, 2004.
- [7] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [8] J. Dean and L. A. Barroso, "The Tail at Scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.
- [9] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2016.
- [10] A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [11] M. Li et al., "Scaling Distributed Machine Learning with the Parameter Server," *Proceedings of OSDI*, 2014.
- [12] B. Recht et al., "Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent," *Advances in Neural Information Processing Systems*, 2011.
- [13] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [14] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [15] K. Bonawitz et al., "Towards Federated Learning at Scale: System Design," *Proceedings of MLSys*, 2019.
- [16] A. Borzunov et al., "Petals: Collaborative Inference and Fine-tuning of Large Models," *Proceedings of ACL (System Demonstrations)*, 2023.
- [17] M. Ryabinin and A. Gusev, "Towards Crowdsourced Training of Large Neural Networks using Decentralized

Mixture-of-Experts," *Advances in Neural Information Processing Systems*, 2020.

[18] G. Gerganov, "llama.cpp: LLM Inference in C/C++," GitHub repository, 2023. [Online]. Available: <https://github.com/ggml-org/llama.cpp>

[19] Holepunch, "Hyperswarm: A Distributed Networking Stack for Connecting Peers," GitHub repository. [Online]. Available: <https://github.com/holepunchto/hyperswarm>

[20] P2P Labs, "Peersky Browser: A Local-First Peer-to-Peer Web Browser," GitHub repository. [Online]. Available: <https://github.com/p2plabsxyz/peersky-browser>